Level 1: Basics

QUESTION 1:

```
#1. Retrieve customer names and emails for email marketing
#This helps the marketing team extract basic customer contact details for campaigns.

select name, email from customers;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

name	email
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com
Leslie Wilson	user13@example.com
Deborah Arias	user14@example.com
Austin Flores	user15@example.com
Amy Landry	user16@example.com
Randy Mooney	user17@example.com
Jeffrey Bray	user18@example.com

QUESTION 2:

```
#2. View complete product catalog with all available details
#The product manager may want to review all product listings in one go.

select * from products;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	category	price	stock_quantity	added_on
1	Plant No	Home	639.43	152	2024-01-30 06:30:53
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
6	If Whatever	Electronics	177.40	64	2024-12-19 10:37:14
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
9	Common Study	Toys	985.19	171	2023-07-20 13:06:42
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
14	Listen Development	Home	296.17	132	2024-10-12 11:49:26
15	Place Low	Electronics	723.97	46	2023-07-05 14:36:07
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53

QUESTION 3:

```
#3. List all unique product categories
#Useful for analyzing the range of departments or for creating filters on the website.

select distinct category from products;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

category
Home
Clothing
Electronics
Toys
Books

QUESTION 4.

```
#4. Show all products priced above ₹1,000
#This helps identify high-value items for premium promotions or pricing strategy reviews.

select * from products where price > 1000;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
11	Build Her	Books	1852.64	150	2024-09-08 01:09:15
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
16	Special Fact	Toys	1094.18	15	2025-03-17 10:42:40
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30

QUESTION 5:

```
#5. Display products within a mid-range price bracket (₹2,000 to ₹5,000)
#A merchandising team might need this to create a mid-tier pricing campaign.


select * from products where price between 2000 and 5000;
```

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
3	Available Answer	Electronics	2529.51	101	2025-04-13 01:11:46
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
13	Full West	Books	2112.33	172	2023-09-15 03:13:38
17	Everything Plant	Books	2496.68	120	2023-10-08 20:11:55
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40

QUESTION 6:

```
#6. Fetch data for specific customer IDs (e.g., from loyalty program list)
#This is used when customer IDs are pre-selected from another system.


SELECT *
FROM customers
WHERE customer_id IN (6, 9, 12) and customer_id is not null;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	name	email	phone	created_at
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
12	Mary Knight	user12@example.com	361-636-3802	2023-08-16 20:05:50
NULL	NULL	NULL	NULL	NULL

QUESTION 7:

```
#7. Identify customers whose names start with the letter 'A'
#Used for alphabetical segmentation in outreach or app display.


select name from customers where name like 'A%';



#8. List electronics products priced under ₹3,000
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Austin Flores
Amy Landry
Amanda Bright
Adrienne Green

QUESTION 8:

```
#8. List electronics products priced under ₹3,000
#Used by merchandising or frontend teams to showcase budget electronics.



SELECT * FROM products WHERE price > 3000;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30
24	Assume Serve	Home	3437.81	10	2024-07-10 05:05:17
25	South Free	Clothing	3543.58	44	2023-07-25 04:13:43
26	Movement Future	Electronics	3502.62	79	2024-01-23 08:39:07
29	Drop Remember	Electronics	4160.45	89	2023-12-15 12:31:11
30	Future Sort	Home	3043.67	100	2025-05-17 06:55:28
33	Involve Officer	Clothing	4244.09	112	2024-03-22 11:36:17

QUESTION 9:

```
#9. Display product names and prices in descending order of price
#This helps teams easily view and compare top-priced items.

SELECT name, price FROM products ORDER BY price DESC;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08
Drop Remember	4160.45
Data Add	4063.38
Action Ask	4017.01
Weight Enter	3875.18

QUESTION 10:

```
#10. Display product names and prices, sorted by price and then by name
#The merchandising or catalog team may want to list products from most expensive to cheapest.
#multiple products have the same price, they should be sorted alphabetically for clarity on st

SELECT name, price FROM products ORDER BY price DESC, name;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	price
Response Indeed	4897.36
Population Social	4813.68
Development System	4801.78
Any Question	4759.28
Fire Often	4734.89
Natural Network	4722.66
Build High	4707.14
Serious Recognize	4523.10
Study Total	4413.68
Real Source	4398.66
Involve Officer	4244.09
Group But	4180.23
Every Amount	4173.60
Move Small	4168.08
Drop Remember	4160.45
Data Add	4063.38
Action Ask	4017.01
Weight Enter	3875.18

Level 2: Filtering and Formatting

QUESTION 1:

```
#1. Retrieve orders where customer information is missing (possibly due to data migration or deletion)
#Used to identify orphaned orders or test data where customer_id is not linked.


select * from customers where customer_id is null ;
```

customer_id	name	email	phone	created_at
NULL	NULL	NULL	NULL	NULL

QUESTION 2:

```
#2. Display customer names and emails using column aliases for frontend readability
#Useful for feeding into frontend displays or report headings that require user-friendly labels.


select name as 'customer name', email as 'customer emails' from customers;
```

customer name	customer emails
Thomas Owens	user1@example.com
Charles Grant	user2@example.com
Kaitlin Richards	user3@example.com
Christina Williams	user4@example.com
David Allen	user5@example.com
Mark Duke	user6@example.com
Briana Wright	user7@example.com
John Bryan	user8@example.com
Jason Thompson	user9@example.com
Shawn Hill	user10@example.com
Walter Jenkins	user11@example.com
Mary Knight	user12@example.com
Leslie Wilson	user13@example.com
Deborah Arias	user14@example.com
Austin Flores	user15@example.com
Amy Landry	user16@example.com
Randy Mooney	user17@example.com
Jeffrey Bray	user18@example.com
Amanda Bright	user19@example.com
Megan Lee	user20@example.com
Jessica Zamora	user21@example.com
Peter Phillips	user22@example.com
Kathryn Mathews	user23@example.com
Brandy Wright	user24@example.com
Cindy Hart	user25@example.com

customers 14

QUESTION 3:

```
#3. Calculate total value per item ordered by multiplying quantity and item price
#This can help generate per-line item bill details or invoice breakdowns.
SELECT *,(quantity * item_price) AS total_value_per_items
FROM order_items;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content: | Fetch rows:

order_item_id	order_id	product_id	quantity	item_price	total_value_per_items
1	1	19	2	4707.14	9414.28
2	2	6	3	177.40	532.20
3	3	9	3	985.19	2955.57
4	3	23	1	2208.99	2208.99
5	4	35	2	4734.89	9469.78
6	5	19	1	4707.14	4707.14
7	5	7	2	4897.36	9794.72
8	6	7	3	4897.36	14692.08
9	6	37	1	1429.45	1429.45
10	6	15	1	723.97	723.97
11	6	35	3	4734.89	14204.67
12	7	30	1	3043.67	3043.67
13	8	10	3	4801.78	14405.34
14	8	5	3	4722.66	14167.98
15	8	31	2	2070.37	4140.74
16	9	7	3	4897.36	14692.08
17	9	44	1	396.11	396.11
18	9	30	3	3043.67	9131.01
19	10	37	3	1429.45	4288.35
20	10	1	1	639.43	639.43
21	10	7	3	4897.36	14692.08
22	10	5	1	4722.66	4722.66
23	11	1	3	639.43	1918.29
24	11	4	3	4759.28	14277.84
25	12	17	2	2496.68	4993.36

QUESTION 4:

```
#4. Combine customer name and phone number in a single column
#Used to show brief customer summaries or contact lists.
SELECT CONCAT(name, ' ', phone) AS combine FROM customers;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

combine
Thomas Owens 142-479-1945
Charles Grant 9153947511
Kaitlin Richards 2073473421
Christina Williams 586-605-5061x06
David Allen (751)456-8289x1
Mark Duke (144)957-2811
Briana Wright 223-833-9635
John Bryan 045.568.0798x27
Jason Thompson 1862659420
Shawn Hill (268)113-3152x7
Walter Jenkins 536-329-0817x71
Mary Knight 361-636-3802
Leslie Wilson +1-256-261-1984
Deborah Arias 811-821-2144x97
Austin Flores 329.901.1576x66
Amy Landry +1-278-019-3748
Randy Mooney (158)927-7313x3
Jeffrey Bray 164.962.0222x92
Amanda Bright 380.981.9798x69
Megan Lee 7044478333
Jessica Zamora 177-816-7773x00
Peter Phillips 389-656-1695
Kathryn Mathews 001-707-722-178
Brandy Wright 853.520.8915x61
Cindy Hart 617.574.8421x41
Victoria Hall 926-840-0016

Result 16

QUESTION 5:

```
#5. Extract only the date part from order timestamps for date-wise reporting
#Helps group or filter orders by date without considering time.
select date(order_date)as date from orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

date
2025-03-02
2024-10-09
2025-05-08
2024-09-19
2025-04-08
2024-10-25
2024-07-29
2024-07-30
2025-06-10
2025-02-16
2025-03-11
2025-01-15
2025-05-12
2024-09-24
2024-07-18
2024-09-09
2024-08-19
2024-06-15
2024-07-06
2024-08-24
2025-03-12
2025-02-21
2025-03-02
2024-06-19
2024-07-24
2025-02-16

QUESTION 6:

```
#6. List products that do not have any stock left
#This helps the inventory team identify out-of-stock items.
select * from products where stock_quantity =0;
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

product_id	name	category	price	stock_quantity	added_on
NULL	NULL	NULL	NULL	NULL	NULL

Level 3: Aggregations

QUESTION 1:

```
#1. Count the total number of orders placed
#Used by business managers to track order volume over time.

select Count(*) total_orders from orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_orders
400

QUESTION 2:

```
#2. Calculate the total revenue collected from all orders
#This gives the overall sales value.

select sum(total_amount) total_revenue from orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

total_revenue
6960973.66

QUESTION 3:

```
#3. Calculate the average order value
#Used for understanding customer spending patterns.

select round(AVG(amount_paid),2) avg_amount from payments;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

avg_amount
17402.43

QUESTION 4:

```
#4. Count the number of customers who have placed at least one order
#This identifies active customers.

SELECT COUNT(DISTINCT customer_id) AS Atleast_one_order
FROM orders;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Atleast_one_order
30

QUESTION 5:

```
#5. Find the number of orders placed by each customer
#Helpful for identifying top or repeat customers.
SELECT customer_id, COUNT(order_id) order_count FROM orders
GROUP BY customer_id order by customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	order_count
1	12
2	17
3	17
4	9
5	14
6	16
7	13
8	10
9	10
10	13
11	9
12	11
13	12
14	15
15	15
16	16
17	17
18	9
19	15
20	17
21	13

QUESTION 6:

```
#6. Find total sales amount made by each customer
SELECT customer_id, sum(total_amount) total_sales
FROM orders GROUP BY customer_id order by customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	total_sales
1	183747.44
2	284420.07
3	253783.31
4	137562.22
5	262504.19
6	212173.58
7	167960.11
8	164701.78
9	106226.03
10	252722.75
11	173409.01
12	169653.79
13	152727.70
14	360324.18
15	260499.91
16	278469.73
17	262189.71
18	199823.08
19	222256.24
20	229435.29
21	205592.25
22	237879.99

QUESTION 7:

```
#7. List the number of products sold per category
#This helps category managers assess performance by department.
SELECT category, COUNT(*) Product_count
FROM products GROUP BY category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	Product_count
Home	8
Clothing	13
Electronics	14
Toys	8
Books	7

QUESTION 8:

```
#8. Find the average item price per category
#Useful to compare pricing across departments.
SELECT category, ROUND(AVG(price), 2) Avg_price
FROM products GROUP BY category;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

category	Avg_price
Home	2146.37
Clothing	3434.58
Electronics	2653.39
Toys	2516.53
Books	3167.08

QUESTION 9:

```
#9. Show number of orders placed per day
#Used to track daily business activity and demand trends.

select date(order_date) Order_date, count(order_id) Order_count
 from orders group by date(order_date) ;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

Order_date	Order_count
2025-03-02	3
2024-10-09	2
2025-05-08	2
2024-09-19	3
2025-04-08	3
2024-10-25	3
2024-07-29	2
2024-07-30	2
2025-06-10	2
2025-02-16	2
2025-03-11	3
2025-01-15	4
2025-05-12	2
2024-09-24	1
2024-07-18	1
2024-09-09	3
2024-08-19	1
2024-06-15	1
2024-07-06	2
2024-08-24	3
2025-03-12	1
2025-02-21	2

QUESTION 10:

```
#10. List total payments received per payment method
#Helps the finance team understand preferred transaction modes.

select method, sum(amount_paid) total from payments group by method;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

method	total
Credit Card	1754603.10
Net Banking	1658383.90
UPI	1617408.78
Debit Card	1930577.88

Level 4: Multi-Table Queries (JOINS)

QUESTION 1:

```
#1. Retrieve order details along with the customer name (INNER JOIN)
#Used for displaying which customer placed each order.

select c.name, o.* from customers c join orders o
on c.customer_id = o.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	order_id	customer_id	order_date	status	total_amount
Thomas Owens	14	1	2024-09-24 21:21:38	Shipped	15803.34
Thomas Owens	17	1	2024-08-19 21:17:57	Pending	11173.77
Thomas Owens	61	1	2024-12-26 23:21:58	Shipped	13053.00
Thomas Owens	76	1	2025-01-14 22:59:54	Pending	23506.81
Thomas Owens	92	1	2024-09-26 11:33:58	Shipped	834.75
Thomas Owens	109	1	2025-03-17 04:56:18	Shipped	12190.14
Thomas Owens	127	1	2025-06-10 18:01:32	Pending	20160.64
Thomas Owens	135	1	2025-03-11 22:58:23	Pending	8891.79
Thomas Owens	144	1	2024-09-19 09:18:22	Shipped	12093.54
Thomas Owens	221	1	2024-09-25 05:49:39	Pending	12437.61
Thomas Owens	278	1	2025-02-01 20:05:38	Delivered	32015.16
Thomas Owens	379	1	2025-03-22 15:57:58	Cancelled	21586.89
Charles Grant	56	2	2024-06-23 15:59:57	Pending	6828.61
Charles Grant	169	2	2025-02-11 04:04:38	Cancelled	9426.30
Charles Grant	209	2	2024-10-16 10:07:48	Shipped	7690.62
Charles Grant	223	2	2024-11-25 01:32:18	Cancelled	41020.75
Charles Grant	230	2	2024-07-25 22:19:17	Pending	22655.32
Charles Grant	232	2	2025-05-02 17:14:21	Cancelled	9583.84
Charles Grant	248	2	2025-06-06 14:36:15	Pending	9829.50
Charles Grant	283	2	2025-03-22 17:37:20	Delivered	14493.79
Charles Grant	315	2	2025-01-27 15:23:27	Cancelled	42056.04
Charles Grant	319	2	2025-04-22 19:16:07	Delivered	1024.92

QUESTION 2:

```
#2. Get list of products that have been sold (INNER JOIN with order_items)
#Used to find which products were actually included in orders.
SELECT DISTINCT p.product_id, p.name
FROM products p
INNER JOIN order_items oi ON p.product_id = oi.product_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	name
1	Plant No
2	Population Social
3	Available Answer
4	Any Question
5	Natural Network
6	If Whatever
7	Response Indeed
8	Every Amount
9	Common Study
10	Development System
11	Build Her
12	Action Ask
13	Full West
14	Listen Development
15	Place Low
16	Special Fact
17	Everything Plant
18	Some Them
19	Build High
20	Real Source
21	Bed Including
22	Move Small

QUESTION 3:

```
#3. List all orders with their payment method (INNER JOIN)
#Used by finance or audit teams to see how each order was paid for.

select o.*, p.method from orders o join payments p on o.order_id = p.order_id;
```

esult Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	customer_id	order_date	status	total_amount	method
1	20	2025-03-02 07:20:11	Delivered	9414.28	Credit Card
2	18	2024-10-09 18:08:21	Shipped	532.20	Net Banking
3	15	2025-05-08 00:08:27	Cancelled	5164.56	Credit Card
4	11	2024-09-19 22:16:13	Delivered	9469.78	UPI
5	12	2025-04-08 18:02:06	Pending	14501.86	UPI
6	29	2024-10-25 07:33:59	Cancelled	31050.17	UPI
7	22	2024-07-29 11:58:47	Shipped	3043.67	UPI
8	19	2024-07-30 22:49:49	Cancelled	32714.06	Net Banking
9	6	2025-06-10 17:00:25	Pending	24219.20	Net Banking
10	28	2025-02-16 12:45:59	Delivered	24342.52	Debit Card
11	25	2025-03-11 14:22:46	Cancelled	16196.13	Credit Card
12	28	2025-01-15 13:22:53	Shipped	9890.72	Credit Card
13	25	2025-05-12 13:04:29	Cancelled	9627.36	UPI
14	1	2024-09-24 21:21:38	Shipped	15803.34	UPI
15	29	2024-07-18 03:18:24	Pending	12421.88	UPI
16	5	2024-09-09 03:00:40	Shipped	22856.73	UPI
17	1	2024-08-19 21:17:57	Pending	11173.77	UPI
18	13	2024-06-15 12:57:04	Cancelled	32001.24	Net Banking
19	29	2024-07-06 15:28:24	Cancelled	843.46	Net Banking
20	20	2024-08-24 04:48:37	Delivered	6400.68	Debit Card
21	22	2025-03-12 18:53:46	Pending	25364.11	Debit Card
22	20	2025-02-21 11:21:06	Shipped	21281.44	Credit Card

QUESTION 4:

```
#4. Get list of customers and their orders (LEFT JOIN)
#Used to find all customers and see who has or hasn't placed orders.
select c.name, o.* from customers c left join orders o
on c.customer_id = o.customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	order_id	customer_id	order_date	status	total_amount
Thomas Owens	14	1	2024-09-24 21:21:38	Shipped	15803.34
Thomas Owens	17	1	2024-08-19 21:17:57	Pending	11173.77
Thomas Owens	61	1	2024-12-26 23:21:58	Shipped	13053.00
Thomas Owens	76	1	2025-01-14 22:59:54	Pending	23506.81
Thomas Owens	92	1	2024-09-26 11:33:58	Shipped	834.75
Thomas Owens	109	1	2025-03-17 04:56:18	Shipped	12190.14
Thomas Owens	127	1	2025-06-10 18:01:32	Pending	20160.64
Thomas Owens	135	1	2025-03-11 22:58:23	Pending	8891.79
Thomas Owens	144	1	2024-09-19 09:18:22	Shipped	12093.54
Thomas Owens	221	1	2024-09-25 05:49:39	Pending	12437.61
Thomas Owens	278	1	2025-02-01 20:05:38	Delivered	32015.16
Thomas Owens	379	1	2025-03-22 15:57:58	Cancelled	21586.89
Charles Grant	56	2	2024-06-23 15:59:57	Pending	6828.61
Charles Grant	169	2	2025-02-11 04:04:38	Cancelled	9426.30
Charles Grant	209	2	2024-10-16 10:07:48	Shipped	7690.62
Charles Grant	223	2	2024-11-25 01:32:18	Cancelled	41020.75
Charles Grant	230	2	2024-07-25 22:19:17	Pending	22655.32
Charles Grant	232	2	2025-05-02 17:14:21	Cancelled	9583.84
Charles Grant	248	2	2025-06-06 14:36:15	Pending	9829.50
Charles Grant	283	2	2025-03-22 17:37:20	Delivered	14493.79
Charles Grant	315	2	2025-01-27 15:23:27	Cancelled	42056.04
Charles Grant	319	2	2025-04-22 19:16:07	Delivered	1024.92

QUESTION 5:

```
#5. List all products along with order item quantity (LEFT JOIN)
#Useful for inventory teams to track what sold and what hasn't.

select p.name, sum(oi.quantity) Order_quantity from products p left join order_items oi
on p.product_id = oi.product_id group by p.name order by Order_quantity desc;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	Order_quantity
Assume Serve	83
Age Treatment	67
Some Them	63
Southern Thing	62
Every Amount	61
Move Small	60
Real Source	60
Between Up	58
Everything Plant	58
Group But	58
Fire Often	58
Common Study	57
Series Page	54
If Whatever	54
Least Green	54
Development System	53
Stock Article	53
Suffer We	53
Life Series	52
Involve Officer	52
Movement Future	51
South Free	51

QUESTION 6:

```
#6. List all payments including those with no matching orders (RIGHT JOIN)
#Rare but used when ensuring all payments are mapped correctly.
select p.* from  orders o right join payments p
on p.order_id = o.order_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

payment_id	order_id	payment_date	amount_paid	method
1	1	2025-03-02 08:16:11	9414.28	Credit Card
2	2	2024-10-09 19:00:21	532.20	Net Banking
3	3	2025-05-08 00:55:27	5164.56	Credit Card
4	4	2024-09-19 22:28:13	9469.78	UPI
5	5	2025-04-08 18:26:06	14501.86	UPI
6	6	2024-10-25 08:16:59	31050.17	UPI
7	7	2024-07-29 12:45:47	3043.67	UPI
8	8	2024-07-30 23:42:49	32714.06	Net Banking
9	9	2025-06-10 17:36:25	24219.20	Net Banking
10	10	2025-02-16 13:43:59	24342.52	Debit Card
11	11	2025-03-11 15:05:46	16196.13	Credit Card
12	12	2025-01-15 14:00:53	9890.72	Credit Card
13	13	2025-05-12 14:01:29	9627.36	UPI
14	14	2024-09-24 22:11:38	15803.34	UPI
15	15	2024-07-18 03:45:24	12421.88	UPI
16	16	2024-09-09 03:38:40	22856.73	UPI
17	17	2024-08-19 21:49:57	11173.77	UPI
18	18	2024-06-15 13:45:04	32001.24	Net Banking
19	19	2024-07-06 15:43:24	843.46	Net Banking
20	20	2024-08-24 04:59:37	6400.68	Debit Card
21	21	2025-03-12 19:48:46	25364.11	Debit Card

QUESTION 7:

```
#7. Combine data from three tables: customer, order, and payment
#Used for detailed transaction reports.
SELECT c.name, o.order_id, o.total_amount, p.method, p.amount_paid
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
JOIN payments p ON o.order_id = p.order_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	order_id	total_amount	method	amount_paid
Thomas Owens	14	15803.34	UPI	15803.34
Thomas Owens	17	11173.77	UPI	11173.77
Thomas Owens	61	13053.00	Net Banking	13053.00
Thomas Owens	76	23506.81	Credit Card	23506.81
Thomas Owens	92	834.75	Net Banking	834.75
Thomas Owens	109	12190.14	UPI	12190.14
Thomas Owens	127	20160.64	Debit Card	20160.64
Thomas Owens	135	8891.79	Debit Card	8891.79
Thomas Owens	144	12093.54	UPI	12093.54
Thomas Owens	221	12437.61	UPI	12437.61
Thomas Owens	278	32015.16	UPI	32015.16
Thomas Owens	379	21586.89	Credit Card	21586.89
Charles Grant	56	6828.61	Credit Card	6828.61
Charles Grant	169	9426.30	Credit Card	9426.30
Charles Grant	209	7690.62	Debit Card	7690.62
Charles Grant	223	41020.75	Debit Card	41020.75
Charles Grant	230	22655.32	Credit Card	22655.32
Charles Grant	232	9583.84	UPI	9583.84
Charles Grant	248	9829.50	Credit Card	9829.50
Charles Grant	283	14493.79	Net Banking	14493.79
Charles Grant	315	42056.04	UPI	42056.04

Level 5: Subqueries (Inner Queries)

QUESTION 1:

```
#1. List all products priced above the average product price
#Used by pricing analysts to identify premium-priced products.

select
* FROM PRODUCTS
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content

product_id	name	category	price	stock_quantity	added_on
2	Population Social	Clothing	4813.68	84	2025-05-30 10:02:50
4	Any Question	Clothing	4759.28	179	2025-06-03 13:34:03
5	Natural Network	Toys	4722.66	75	2023-11-06 00:47:37
7	Response Indeed	Clothing	4897.36	36	2025-03-29 02:43:08
8	Every Amount	Home	4173.60	156	2025-04-30 03:11:10
10	Development System	Electronics	4801.78	153	2025-03-12 08:22:57
12	Action Ask	Electronics	4017.01	19	2025-02-14 03:38:06
18	Some Them	Toys	3673.86	110	2024-07-25 00:33:53
19	Build High	Clothing	4707.14	47	2023-09-01 02:50:01
20	Real Source	Books	4398.66	197	2025-02-08 10:28:27
21	Bed Including	Clothing	3845.31	92	2025-01-21 11:52:40
22	Move Small	Books	4168.08	22	2023-11-12 18:26:30

QUESTION 2:

```
#2. Find customers who have placed at least one order
#Used to identify active customers for loyalty campaigns.
SELECT * FROM CUSTOMERS
WHERE CUSTOMER_ID IN (SELECT CUSTOMER_ID FROM ORDERS);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

customer_id	name	email	phone	created_at
1	Thomas Owens	user1@example.com	142-479-1945	2024-10-14 16:01:12
2	Charles Grant	user2@example.com	9153947511	2023-11-25 15:45:24
3	Kaitlin Richards	user3@example.com	2073473421	2024-06-23 09:55:22
4	Christina Williams	user4@example.com	586-605-5061x06	2024-10-27 17:19:38
5	David Allen	user5@example.com	(751)456-8289x1	2023-10-29 02:43:00
6	Mark Duke	user6@example.com	(144)957-2811	2024-06-24 03:22:59
7	Briana Wright	user7@example.com	223-833-9635	2023-06-25 00:35:43
8	John Bryan	user8@example.com	045.568.0798x27	2025-02-15 17:57:04
9	Jason Thompson	user9@example.com	1862659420	2024-08-31 08:18:51
10	Shawn Hill	user10@example.com	(268)113-3152x7	2023-12-14 20:46:43
11	Walter Jenkins	user11@example.com	536-329-0817x71	2023-10-26 03:12:30
12	Mary Knight	user12@example.com	361-636-3802	2023-08-16 20:05:50
13	Leslie Wilson	user13@example.com	+1-256-261-1984	2024-06-06 20:12:35
14	Deborah Arias	user14@example.com	811-821-2144x97	2024-04-24 00:27:28
15	Austin Flores	user15@example.com	329.901.1576x66	2024-06-13 09:03:42
16	Amy Landry	user16@example.com	+1-278-019-3748	2024-02-28 17:51:50
17	Randy Mooney	user17@example.com	(158)927-7313x3	2023-09-21 13:23:02
18	Jeffrey Bray	user18@example.com	164.962.0222x92	2024-07-02 19:51:55
19	Amanda Bright	user19@example.com	380.981.9798x69	2024-12-20 22:58:15
20	Megan Lee	user20@example.com	7044478333	2024-07-09 16:13:14
21	Jessica Zamora	user21@example.com	177-816-7773x00	2023-11-11 04:46:03
22	Peter Phillips	user22@example.com	389-656-1695	2025-05-22 06:12:06
23	Kathryn Mathews	user23@example.com	001-707-722-178	2023-12-21 15:30:24
24	Brandy Wright	user24@example.com	853.520.8915x61	2024-07-09 17:09:19
25	Cindy Hart	user25@example.com	617.574.8421x41	2025-05-27 18:02:54

QUESTION 3:

```
#3. Show orders whose total amount is above the average for that customer
#Used to detect unusually high purchases per customer.

SELECT order_id, total_amount, customer_id
FROM orders o1
WHERE total_amount > (SELECT AVG(total_amount) FROM orders o2 WHERE o2.customer_id = o1.customer_id);
```

Result Grid | Filter Rows: | Edit: | Export/Import: | Wrap Cell Content:

order_id	total_amount	customer_id
6	31050.17	29
8	32714.06	19
9	24219.20	6
10	24342.52	28
14	15803.34	1
16	22856.73	5
18	32001.24	13
21	25364.11	22
22	21281.44	20
24	22882.19	6
25	23749.66	17
26	35167.92	24
29	26308.25	7
30	18628.74	23
32	23536.35	15
33	36147.09	8
36	34125.89	25
40	24649.49	26
42	21622.22	3
44	17686.50	12
45	27016.65	19
46	24408.08	29
48	28719.50	28

QUESTION 4:

```
#4. Display customers who haven't placed any orders
#Used for re-engagement campaigns targeting inactive users.

SELECT name, email
FROM customers
WHERE customer_id NOT IN (SELECT customer_id FROM orders );
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	email

QUESTION 5:

```
#5. Show products that were never ordered
#Helps with inventory clearance decisions or product deactivation.

SELECT NAME FROM PRODUCTS WHERE PRODUCT_ID NOT IN ( SELECT PRODUCT_ID FROM ORDER_ITEMS);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

NAME

QUESTION 6:

```
#6. Show highest value order per customer
#Used to identify the largest transaction made by each customer.

SELECT customer_id, MAX(total_amount) AS highest_value
FROM orders
GROUP BY customer_id;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

customer_id	highest_value
1	32015.16
2	42056.04
3	41679.11
4	25747.34
5	39921.78
6	39003.19
7	28589.04
8	36147.09
9	21414.44
10	35723.17
11	37129.82
12	38656.26
13	32001.24
14	48042.06
15	40510.54
16	37415.67
17	40944.10
18	39857.19
19	35676.00
20	39563.51
21	24096.64
22	36504.68

QUESTION 7:

```sql
#7. Highest Order Per Customer (Including Names)
#Used to identify the largest transaction made by each customer. Outputs name as well.

SELECT c.name, o.total_amount
FROM customers c
JOIN orders o ON c.customer_id = o.customer_id
WHERE o.total_amount = (SELECT MAX(total_amount) FROM orders o2 WHERE o2.customer_id = c.customer_id);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name	total_amount
Thomas Owens	32015.16
Charles Grant	42056.04
Kaitlin Richards	41679.11
Christina Williams	25747.34
David Allen	39921.78
Mark Duke	39003.19
Briana Wright	28589.04
John Bryan	36147.09
Jason Thompson	21414.44
Shawn Hill	35723.17
Walter Jenkins	37129.82
Mary Knight	38656.26
Leslie Wilson	32001.24
Deborah Arias	48042.06
Austin Flores	40510.54
Amy Landry	37415.67
Randy Mooney	40944.10
Jeffrey Bray	39857.19
Amanda Bright	35676.00
Megan Lee	39563.51
Jessica Zamora	24096.64
Peter Phillips	36504.68

Level 6: Set Operations

QUESTION 1:

```
#1. List all customers who have either placed an order or written a product review
#Used to identify engaged customers from different activity areas.

SELECT name
FROM customers
WHERE customer_id IN (SELECT customer_id FROM orders
UNION
SELECT customer_id FROM product_reviews);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Thomas Owens
Charles Grant
Kaitlin Richards
Christina Williams
David Allen
Mark Duke
Briana Wright
John Bryan
Jason Thompson
Shawn Hill
Walter Jenkins
Mary Knight
Leslie Wilson
Deborah Arias
Austin Flores
Amy Landry
Randy Mooney
Jeffrey Bray
Amanda Bright
Megan Lee
Jessica Zamora

QUESTION 2:

```
#2. List all customers who have placed an order as well as reviewed a product [intersect
#Used to identify highly engaged customers for rewards.

SELECT name FROM customers WHERE customer_id IN (SELECT customer_id FROM orders
    INTERSECT SELECT customer_id FROM product_reviews);
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

name
Thomas Owens
Charles Grant
Christina Williams
Mark Duke
Briana Wright
Jason Thompson
Shawn Hill
Walter Jenkins
Leslie Wilson
Deborah Arias
Austin Flores
Randy Mooney
Amanda Bright
Megan Lee
Jessica Zamora
Peter Phillips
Kathryn Mathews
Cindy Hart
Victoria Hall
Adrienne Green
Joseph Stuart
Kara Zavala

Result 144